Title: Employee ID Management Using BST

1. Problem Statement:

Employee records with unique IDs must be stored and retrieved quickly. In arrays or unsorted linked lists, searching an ID takes O(n) time, sorted arrays need costly shifting on insertion, and duplicate IDs are not prevented. This project implements a C++ program using a Binary Search Tree (BST) to insert, search and display employee records efficiently, reject duplicate IDs and show records in sorted order.

2. Objectives:

- To implement a BST in C++ that stores employee IDs and names.
- To insert records in the correct position and reject duplicate IDs.
- To search an employee by ID and count the comparisons made.
- To display all records in ascending order using inorder traversal.

3. Algorithm:

Step 1: Start the program.

Step 2: Initialize the root pointer of the BST to NULL.

Step 3: Display the menu (1. Insert Employee, 2. Search Employee, 3. Display Employees, 4. Exit) and read the choice.

Step 4: Execute the control flow based on the choice:

• If choice == 1 (Insert): Read the employee ID and name. If root is NULL, create a new node and make it the root. Otherwise start from the root: if the ID is smaller than the current node's ID move to the left child, if it is greater move to the right child, and if it is equal display a duplicate ID message and stop. When a NULL position is reached, create the new node and attach it there.

• If choice == 2 (Search): Read the employee ID, set comparisons = 0 and temp = root. While temp is not NULL, increase comparisons by 1; if the ID matches, display the employee name and stop; otherwise move to the left child if the ID is smaller or to the right child if it is greater. If temp becomes NULL, display that the employee was not found. Display the number of comparisons.

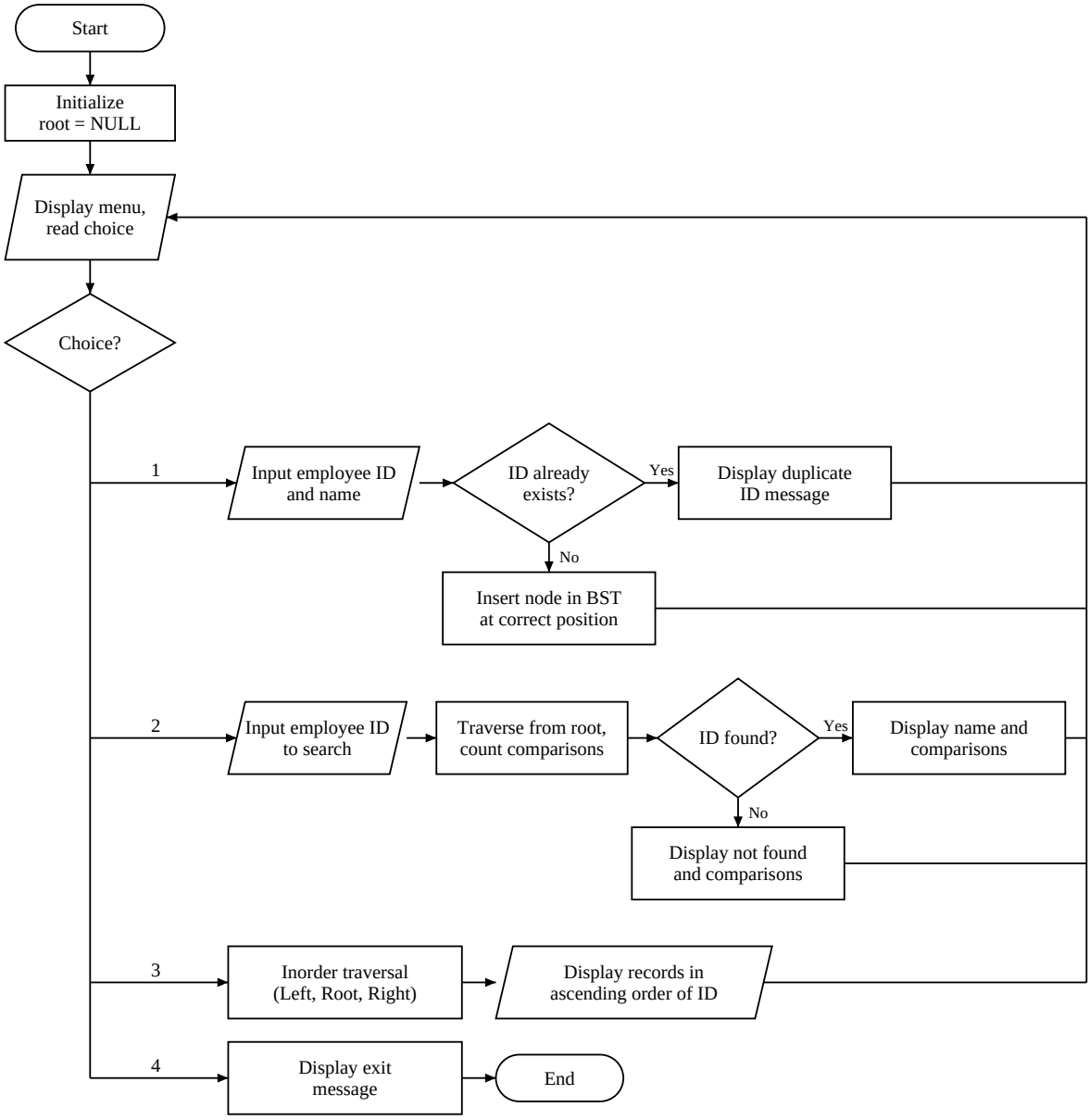
• If choice == 3 (Display): Perform inorder traversal starting from the root: traverse the left subtree, display the current node, then traverse the right subtree. This prints the records in ascending order of ID.

• If choice == 4 (Exit): Display an exit message and go to Step 6.

Step 5: Repeat from Step 3.

Step 6: Stop the program.

4. Flowchart:



5. Conclusion:

The Employee ID Management System using a Binary Search Tree in C++ provides an organized method for storing and managing employee records. It stores employee names along with unique IDs and supports insertion, searching, and displaying records in ascending order.

The inorder traversal technique ensures that employee records are displayed in sorted order. Counting comparisons during searching helps analyze the efficiency of the search operation. The system also prevents duplicate employee IDs, maintaining the uniqueness of employee records.

This project demonstrates the practical application of Binary Search Trees in data management and improves understanding of dynamic data structures, tree traversal, searching algorithms, and algorithm efficiency.